

■ Retos de la Calculadora (con espacio para respuestas)

Hoja de actividad para que los alumnos respondan directamente.

■ Retos de comprensión

- Explica qué es un Token y da un ejemplo de cómo aparece en la expresión ``3 + 5 * 2``.
- ¿Qué diferencia hay entre el Lexer y el Parser?
- ¿Qué significa que el parser sea recursivo? Pon un ejemplo de función que lo demuestra.

■ Retos de depuración

- ¿Qué devuelve la calculadora con la entrada: ``2 + 3 * 4`` y por qué?
- ¿Qué ocurre si escribimos una expresión no válida como ``2 + *``? ¿Cómo reacciona el código?
- ¿Qué devuelve si calculamos ``(2 + 3) ^ 2``?

■ Retos de modificación

- Añade la función ``tan(x)`` usando el mismo esquema que ``sin`` y ``cos``.
- Modifica el parser para que acepte también la función ``sqrt(x)``.
- Haz que la calculadora acepte números negativos explícitos como ``-5 + 3``.

■ Retos de predicción

- ¿Qué resultado debería devolver esta expresión y por qué?: ``cos(0) + sin(90)`` (recuerda: las funciones trigonométricas usan radianes).
- ¿Cuál es el resultado de ``2 ^ 3 ^ 2``? Explica por qué según la precedencia implementada.
- ¿Qué devuelve la calculadora con ``(2 + 3) * (4 + 5)``?

■ Retos de diseño

- El código actual usa un parser recursivo. ¿Qué ventaja tiene frente a procesar los tokens con un bucle y pila manual?
- ¿Por qué es útil separar la calculadora en las fases lexer → parser → evaluator en vez de hacerlo todo en un solo método?
- Si quisieras añadir soporte para variables (`x = 5`, `y = 2 * x`), ¿en qué parte del código lo implementarías y por qué?